Outline

- Types of Verification
  - Dynamic Verification
  - Static Verification
- Software Testing
  - Definition
  - Testing Processes
  - Testing Levels
  - Testing Guidance

Types of Verification

- **Dynamic Verification** (e.g., software testing):
  - Software implementation is executed with test data.
  - Applied to a prototype or an executable version.
  - Actual vs expected outputs are compared.

- **Static Verification**:
  - Analyze and check system representations such as requirements, specifications, design, and source code.
  - Can be applied at all stages of the software process
  - Walkthrough, inspections, desk checking, correctness proofs, and automated static analyzers.

Definitions

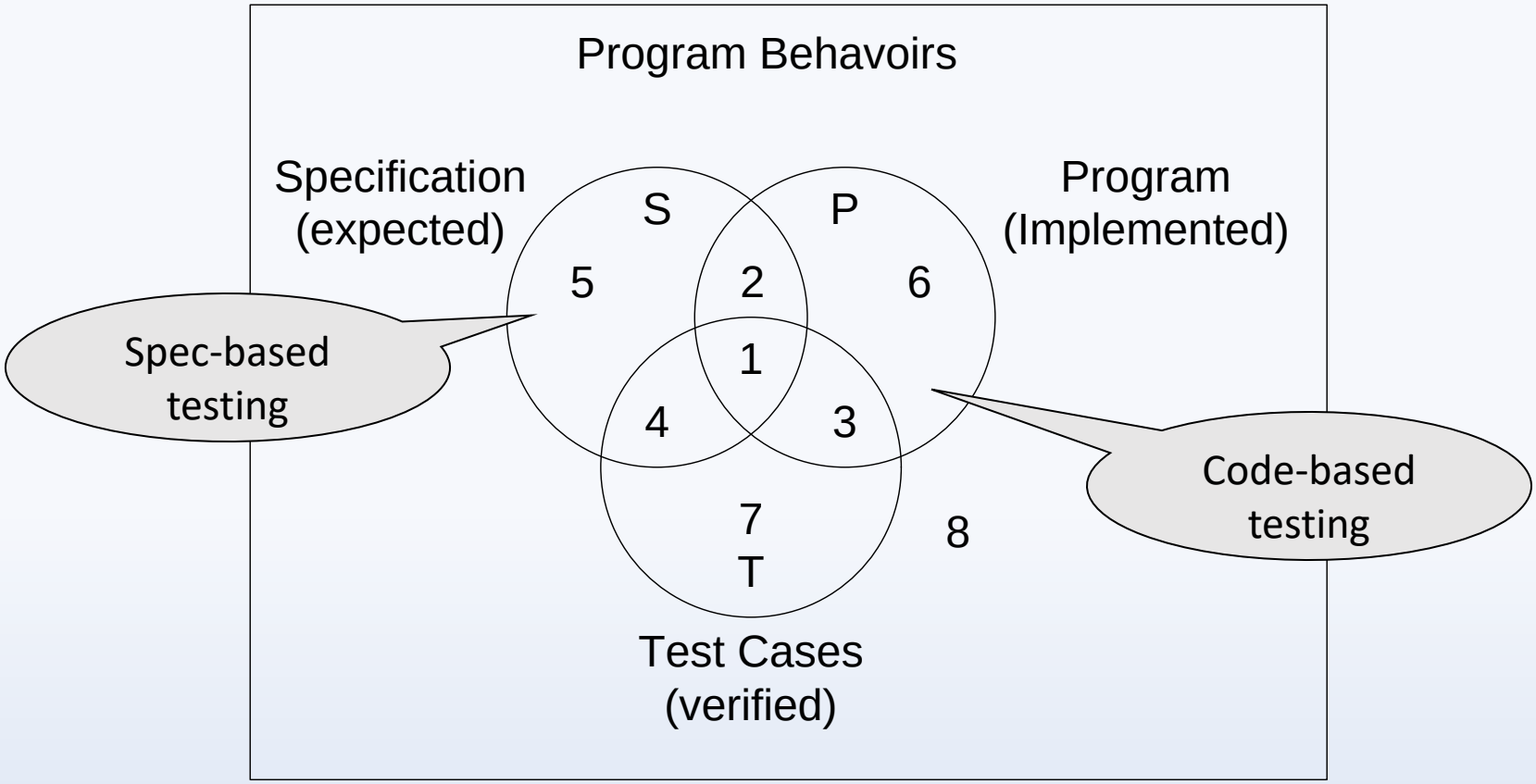
- **Error** – People make errors. A good synonym is a *mistake*.
- **Fault** – The result of an **error**, regardless of if it has ever occurred. May also be called a *defect* or *bug*.
- **Failure** – Occurs when code corresponding to a **fault** executes.
- **Incident** – A symptom of a **failure** that is noticeable to the user.
- **Test Case** – A set of inputs and outputs related to expected program behavior.
- **Test** – The exercising of software with **test cases**, or a collection of **test cases** (depending on context).

Faults and Failures

- Can a fault of omission become a failure?

A. Yes, due to a specification-based test.
B. Yes, due to a code-based test.
C. Nope.
D. None of the above.

Where is the fault of omission?

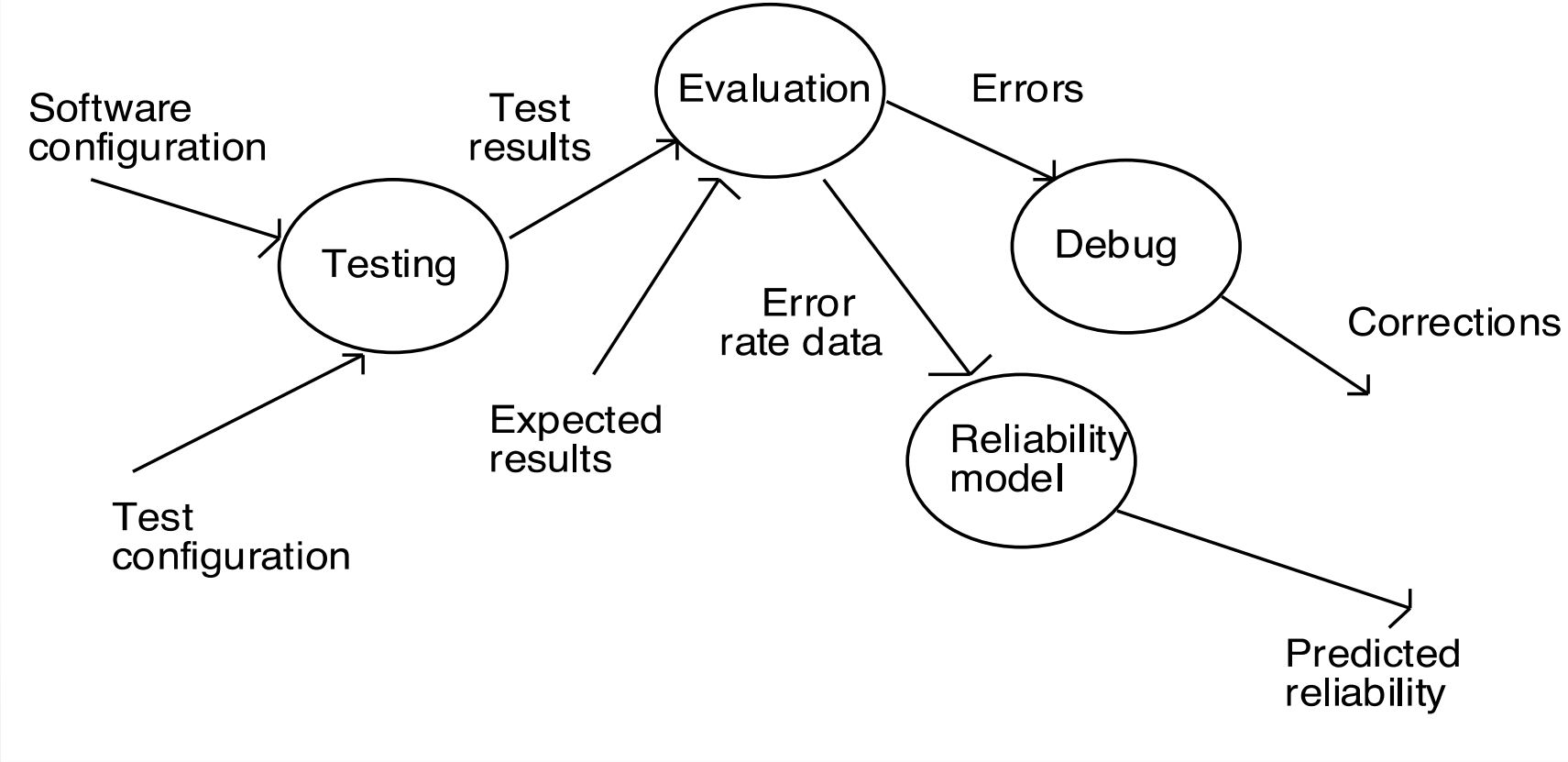


Testing as an Experiment

- With respect to the Error, Fault, Failure, Incident framework, a test case is an experiment that...
  - is designed to **anticipate an error**
  - that is **realized as a fault**
  - that **causes a failure**
  - and is **recognized** by comparing expected and observed outputs.

- In this framework, test cases are intended to reveal incidents.

Testing as an Experiment



Software Configuration:

- Requirements specifications, design specifications, and source code

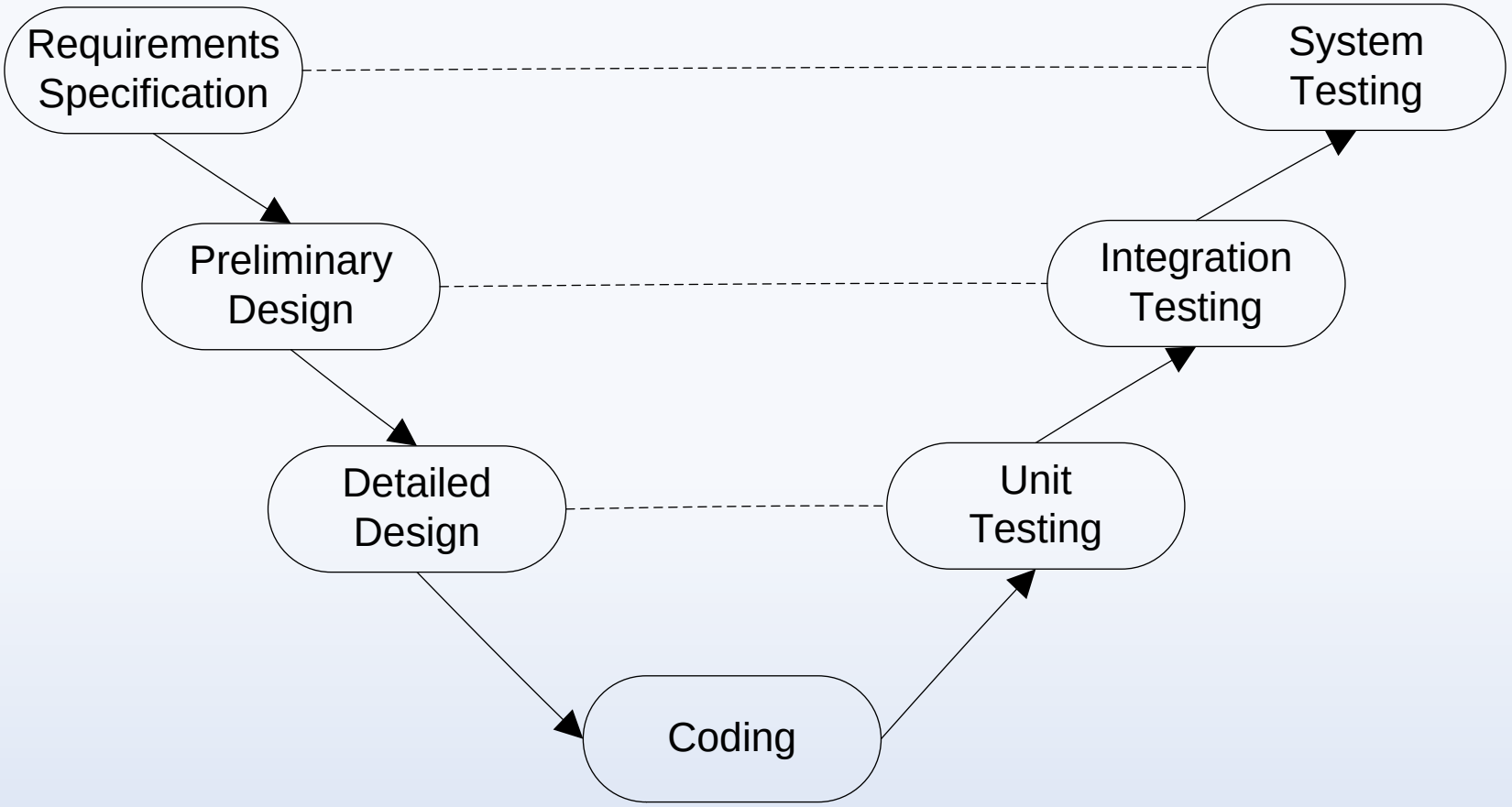
Test Configuration:

- Test plans, procedures, and tools
- Test cases (test data and expected results)

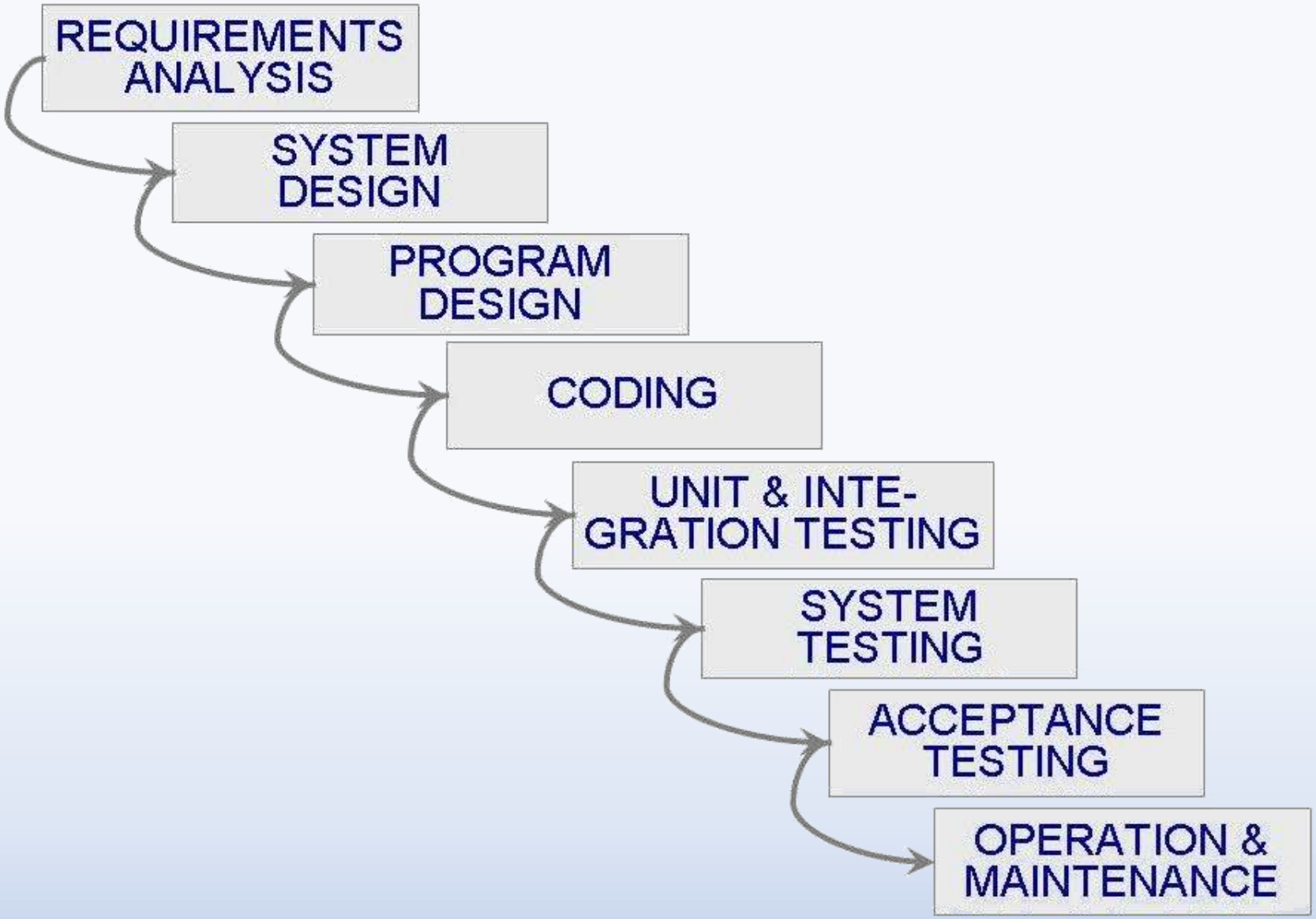
How Might a Test Case Fail / Pass?

	Error Exists	**No Error Exists**
Test Case Fails	Correct!	False Positive!
Test Case Succeeds	False Negative!	Correct!

Levels of Testing



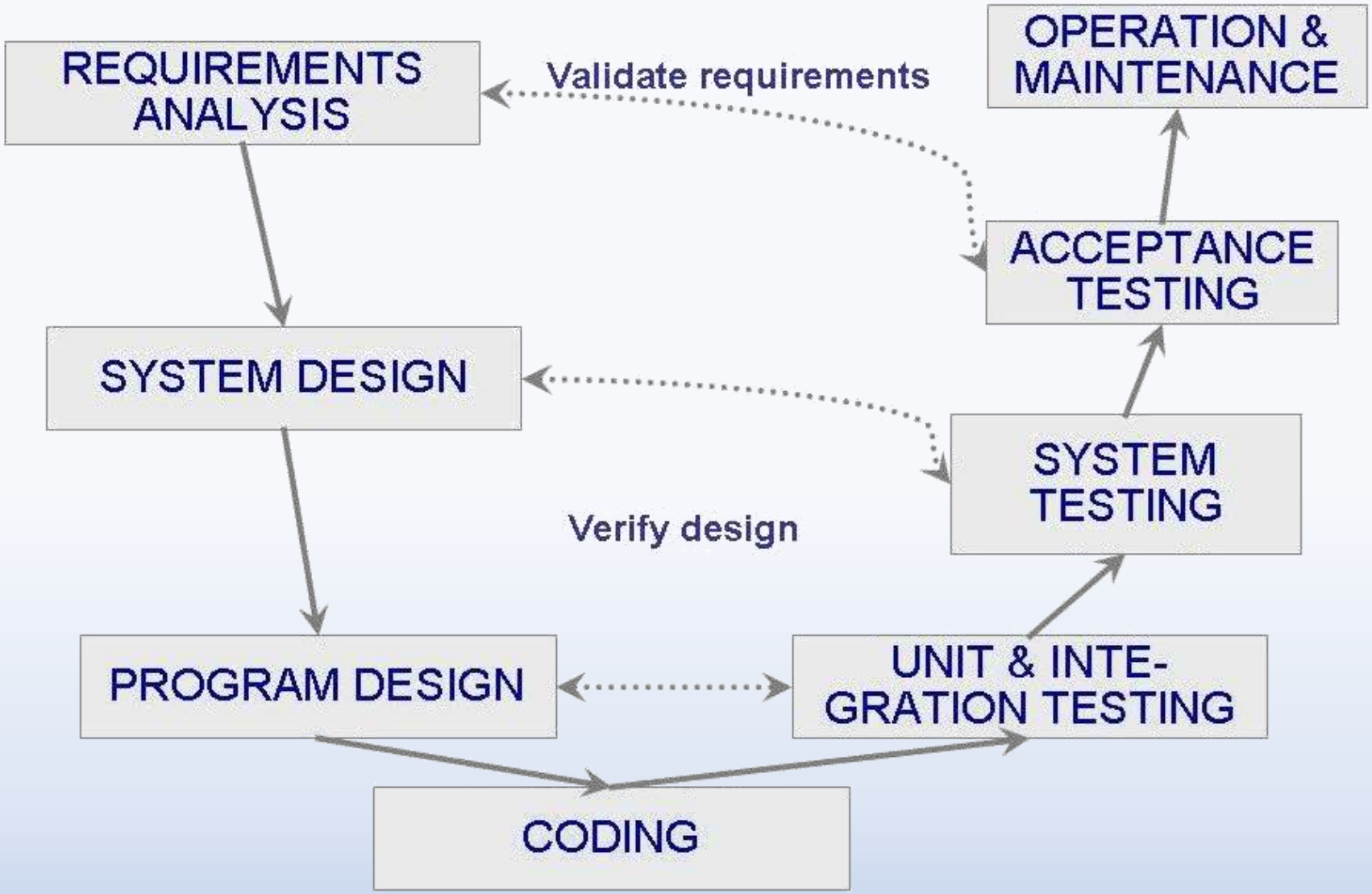
SideBar: Waterfall Software Process Model



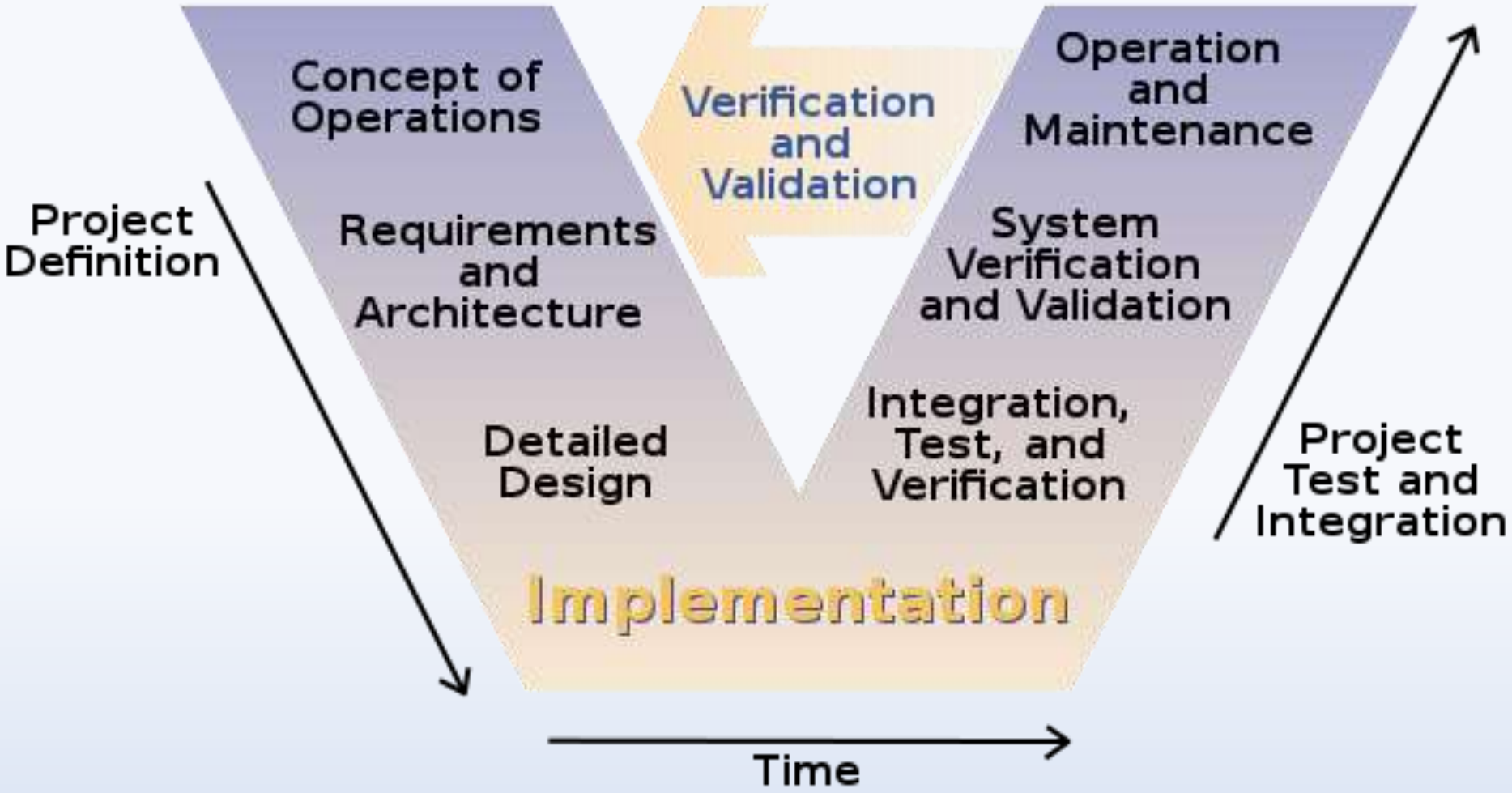
SideBar: Software Process Models: V Model

- A variation of the waterfall model
- Uses unit testing to verify procedural design
- Uses integration testing to verify architectural (system) design
- Uses acceptance testing to validate the requirements
- If problems are found during verification and validation, the left side of the V can be re-executed before testing on the right side is re-enacted

SideBar: Software Process Models: V Model

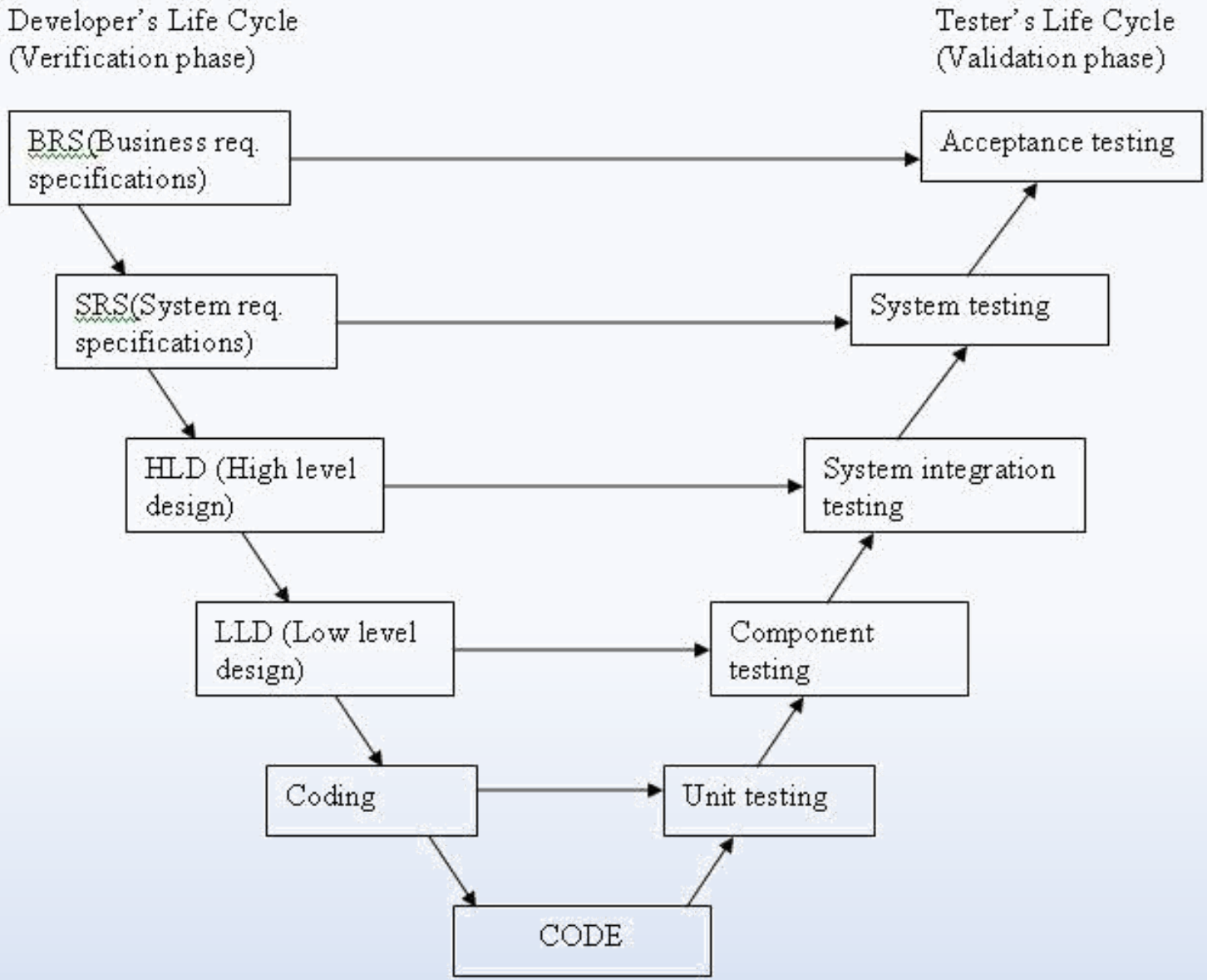


SideBar: Software Process Models: V Model



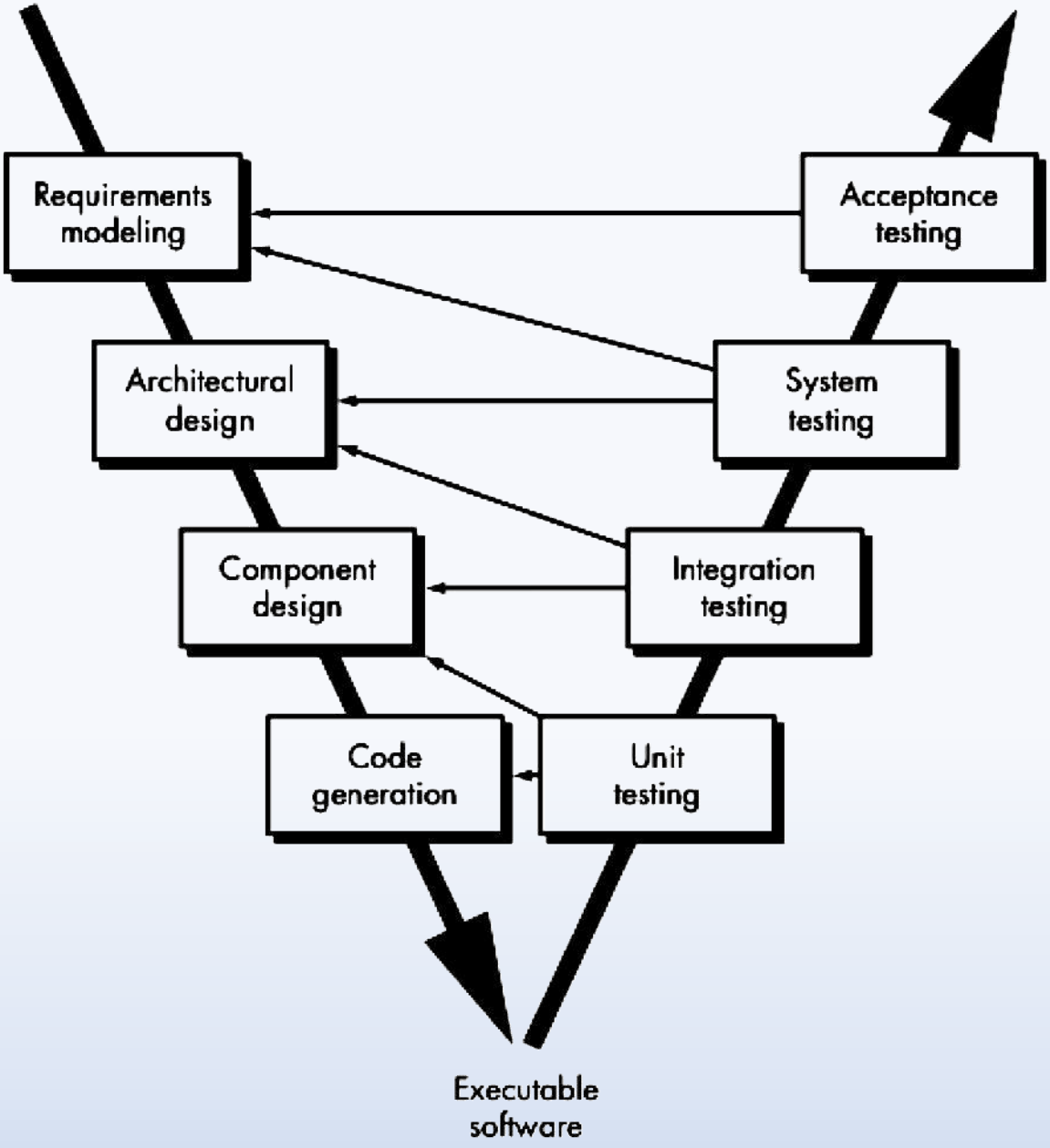
*Clarus Concept of Operations. Publication No. FHWA-JPO-05-072, Federal Highway Administration (FHWA), 2005

SideBar: Software Process Models: V Model



* http://istqbexamcertification.com/what-is-v-model-advantages-disadvantages-and-when-to-use-it/

SideBar: Software Process Models: V Model



* Pressman, Roger S. *Software engineering: a practitioner's approach*

SideBar: Agile Methods: eXtreme Programming

- Emphasis on four characteristics of agility
  - *Communication*: continual interchange between customers and developers
  - *Simplicity*: select the simplest design or implementation
  - *Courage*: commitment to delivering functionality early and often
  - *Feedback*: loops built into the various activities during the development process

Testing Guidelines

- **Testing Guidelines / Principles**:*
  1. A necessary part of a test case is a definition of the expected output or result.
  2. A programmer should avoid attempting to test his or her own program.
  3. A programming organization should not test its own programs.
  4. Thoroughly inspect the results of each tests.
  5. Test cases must be written for input conditions that are invalid and unexpected, as well as for those that are valid and expected.

*As defined by Glenford J. Myers in *The Art of Software Testing*

Testing Guidelines

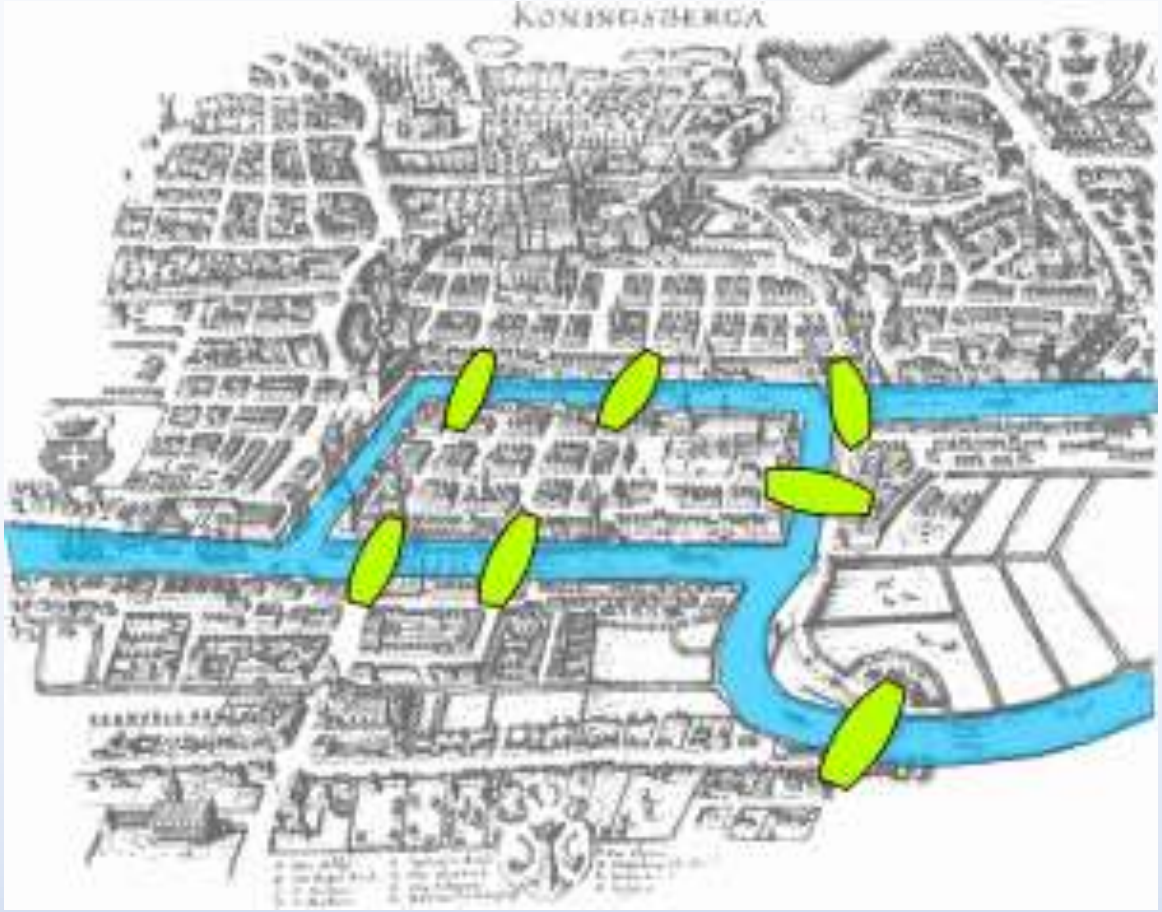
- **Testing Guidelines / Principles**:*
  6. Examining a program to see if it does not do what it is supposed to do is only half the battle; the other half is seeing whether the program does what it is not supposed to do.
  7. Avoid throwaway test cases unless the program is truly a throwaway program.
  8. Do not plan a testing effort under the tacit assumption that no errors will be found.
  9. The probability of the existence of more errors in a section of a program is proportional to the number of errors already in that section.
  10. Testing is an extremely creative and intellectually challenging task.

*As defined by Glenford J. Myers in *The Art of Software Testing*

CIS 613:
Graph Theory for Testers

Bridges of Königsberg Puzzle

- Take a walk around the city, traversing each bridge *exactly once*



https://en.wikipedia.org/wiki/Seven_Bridges_of_Königsberg

Class Overview

In Class:

- Class Introduction (What we just finished)
- Break (10 minutes)
- Software Testing Introduction
- Break (10 minutes)
- Software Testing Overview
- Background Surveys

After Class:

- Office Hours (until 9:30pm or the last person leaves, whichever comes first)

Homework:

- 01-21: Syllabus Quiz
- 01-21: Unit Testing Frameworks
- ***Optional***: Read Software Testing Ch 1, 5, and 6

Syllabus Quiz in Blackboard

- You must complete this quiz with a 100% before any other assignment will be considered submitted.
- You may take this quiz as many times as needed.
- If you find an error on this quiz, please contact the instructor via email